

Documentación Técnica Detallada: Sistema de Simulación y Optimización de Ruido en Edificios

Manual Técnico y de Usuario

19 de enero de 2025

Índice

1. Introducción	2
1.1. Descripción General del Sistema	2
2. Estructura del Sistema	2
2.1. Carga Inicial del Edificio	2
2.2. Modelo de Nodos	3
2.3. Modelado de Propagación del Ruido	4
2.3.1. Atenuación por Distancia	4
2.3.2. Factor de Frecuencia	4
3. Algoritmo de Optimización	4
3.1. Recocido Simulado	4
3.2. Parámetros Principales	5
3.3. Proceso de Optimización	5
3.3.1. 1. Selección de Nodos	5
3.3.2. 2. Matriz de Compatibilidad	6
3.3.3. 3. Proceso de Intercambio	6
4. Visualización y Sistema de Reportes	6
4.1. Visualización 3D	6
4.1.1. Construcción del Grafo	7
4.1.2. Código de Colores	7

5. Sistema de Reportes	7
5.1. Generación de Reportes	7
5.1.1. 1. Reporte Inicial	7
5.1.2. 2. Reporte de Solución	8
6. Interfaz de Usuario	8
6.1. Ventana Principal	8

1. Introducción

1.1. Descripción General del Sistema

Este sistema permite simular y optimizar la distribución de espacios en un edificio considerando la propagación del ruido. El programa:

- Lee una configuración inicial desde un archivo JSON
- Simula la propagación física del ruido entre espacios
- Optimiza la distribución para minimizar problemas acústicos
- Genera visualizaciones 3D interactivas
- Produce reportes detallados con recomendaciones

2. Estructura del Sistema

2.1. Carga Inicial del Edificio

El sistema inicia cargando la configuración del edificio desde un archivo JSON que contiene:

- **Datos de los espacios:**
 - Nombre y tipo de espacio
 - Posición (x,y,z) en el edificio
 - Nivel de piso
 - Características físicas (paredes, ventanas, puertas)
 - Niveles de ruido base
 - Frecuencias características

■ Proceso de carga:

```
1     def inicializar_nodos(self):
2         try:
3             with open('edificio.json', 'r', encoding='utf
4                 -8') as f:
5                 data = json.load(f)
6         except FileNotFoundError:
7             QMessageBox.critical(self, "Error",
8                 "El archivo 'edificio.json' no se encontr
9                 \'{o}.'.")
10            sys.exit(1)
```

2.2. Modelo de Nodos

Cada espacio se representa como un nodo con las siguientes características:

■ Atributos Físicos:

- name: Identificador único del espacio
- tipo: Categoría (aula, biblioteca, etc.)
- position: Coordenadas (x,y,z)
- piso: Nivel en el edificio
- pared: Presencia de paredes (booleano)
- ventana: Presencia de ventanas (booleano)
- puerta: Presencia de puertas (booleano)

■ Atributos Acústicos:

- ruido: Nivel base en decibelios (dB)
- frecuencia: Frecuencia característica (Hz)
- es_fuente: Si es generador activo de ruido

■ Conexiones:

- Lista de nodos vecinos
- Conexiones bidireccionales automáticas
- Sensores de medición asociados

2.3. Modelado de Propagación del Ruido

El cálculo del ruido total en cada espacio sigue un modelo físico que considera múltiples factores:

$$R_{total} = R_{base} + \sum_{i=1}^n (R_i \times A_i \times F_i \times E_i) \quad (1)$$

Donde cada término representa:

- R_{base} : Ruido propio del espacio (si es fuente)
- R_i : Ruido del vecino i
- A_i : Atenuación por distancia según ley cuadrática inversa
- F_i : Factor de atenuación por frecuencia
- E_i : Efectos estructurales combinados

2.3.1. Atenuación por Distancia

La atenuación sigue la ley cuadrática inversa:

$$A_d = \frac{1}{d^2} \quad (2)$$

Donde d es la distancia euclidiana entre espacios:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2} \quad (3)$$

2.3.2. Factor de Frecuencia

La atenuación por frecuencia se calcula como:

$$F_f = \min(1, 0, \frac{1000}{f}) \quad (4)$$

Donde f es la frecuencia en Hz. Este factor modela la mayor atenuación de altas frecuencias.

3. Algoritmo de Optimización

3.1. Recocido Simulado

El algoritmo utiliza recocido simulado para optimizar la distribución de espacios. Este proceso imita el enfriamiento gradual de materiales para encontrar un estado óptimo.

3.2. Parámetros Principales

- **Temperatura Inicial (T_0): 100.0**
 - Controla la probabilidad de aceptar soluciones peores
 - Valor alto inicial permite exploración amplia
- **Factor de Enfriamiento (α): 0.95**
 - Reduce gradualmente la temperatura
 - $T_{nueva} = T_{actual} \times 0,95$
- **Iteraciones: 1000**
 - Número de intentos de optimización
 - Balance entre tiempo y calidad de solución

3.3. Proceso de Optimización

3.3.1. 1. Selección de Nodos

- **Exclusiones Específicas:**
 - Pasillos: No se consideran para intercambio
 - Recepción: Mantiene su ubicación fija
 - Razón: Son espacios estructurales críticos
- **Nodos Problemáticos:**
 - Se identifican espacios que exceden límites de ruido
 - 70 % de probabilidad de seleccionar nodo problemático
 - 30 % de selección aleatoria para diversidad

```
1     nodos_problematicos = [  
2         nodo for nodo in nuevo_grafo.values()  
3         if nodo.tipo != "pasillo" and  
4           nodo.tipo != "recepcion" and  
5           nodo.medir_ruido(vecinos) >  
6           nodo.get_limite_ruido()['limite_cercano']  
7     ]  
8
```

3.3.2. 2. Matriz de Compatibilidad

Define qué tipos de espacios pueden intercambiarse:

Espacio	Compatible con	Justificación
Biblioteca	Oficina, Reuniones	Espacios que requieren silencio relativo
Aula	Laboratorio, Auditorio	Espacios educativos con niveles similares
Cafetería	Reuniones	Espacios sociales con ruido moderado
Laboratorio	Aula, Oficina	Espacios de trabajo controlado
Oficina	Biblioteca, Reuniones	Ambientes de trabajo tranquilos
Auditorio	Aula	Espacios grandes con acústica similar

3.3.3. 3. Proceso de Intercambio

Cuando se seleccionan dos nodos compatibles:

1. Intercambio de Propiedades:

- Tipo de espacio
- Nivel de ruido base
- Condición de fuente
- Posición física

2. Evaluación del Cambio:

- Se calcula nuevo puntaje de habitabilidad
- Se compara con estado anterior

```
1  # Intercambio de propiedades
2  nodo1.tipo, nodo2.tipo = nodo2.tipo, nodo1.tipo
3  nodo1.ruido, nodo2.ruido = nodo2.ruido, nodo1.ruido
4  nodo1.es_fuente, nodo2.es_fuente = nodo2.es_fuente,
   nodo1.es_fuente
5  nodo1.position, nodo2.position = nodo2.position,
   nodo1.position
6
```

4. Visualización y Sistema de Reportes

4.1. Visualización 3D

La visualización del edificio se realiza mediante la clase `Grafo3DWindow`, que implementa:

4.1.1. Construcción del Grafo

■ Creación de Nodos:

```
1 G = nx.Graph()
2 for name, nodo in habitaciones.items():
3     G.add_node(name, tipo=nodo.tipo,
4                 ruido=nodo.medir_ruido(vecinos))
5
```

■ Establecimiento de Conexiones:

1. **Por Piso:** Conecta todos los nodos del mismo nivel
2. **Entre Pisos:** Conecta nodos si la distancia ≤ 7 unidades
3. **Visualización:** Líneas punteadas con grosor inversamente proporcional a la distancia

4.1.2. Código de Colores

Color	Estado	Significado
Verde	Adecuado	Nivel de ruido dentro de límites
Amarillo	Cerca	Próximo a exceder límites
Rojo	Excede	Supera límites permitidos

5. Sistema de Reportes

5.1. Generación de Reportes

El sistema implementa dos tipos de reportes:

5.1.1. 1. Reporte Inicial

```
1 def generar_reporte_inicial(self, grafo):
2     datos_reporte = []
3     for name, nodo in grafo.items():
4         nivel = nodo.medir_ruido(vecinos)
5         estado = determinar_estado(nivel, limites)
6         recomendacion = obtener_recomendacion(
7             tipo_espacio, estado)
8         datos_reporte.append(
9             (name, nivel, estado, recomendacion))
```

Recomendaciones Específicas por Tipo:

■ Biblioteca:

- Excede: Implementar zonas de silencio...”
- Cerca: Reforzar políticas de silencio...”
- Adecuado: Continuar con políticas actuales...”

[Se incluyen todos los tipos de espacios con sus recomendaciones específicas]

5.1.2. 2. Reporte de Solución

■ Contenido:

- Lista de cambios realizados
- Estado final de cada espacio
- Recomendaciones de mejora
- [X] Rojo: Excede límites
- [!] Amarillo: Cerca del límite
- [V] Verde: Nivel adecuado

6. Interfaz de Usuario

6.1. Ventana Principal

■ Botones Principales:

1. Análisis Inicial
2. Solucionar Grafo
3. Salir